LAB 08 - PROMPTS

Execution Stage: an Order Manager that Trades the Paper Account in Any Session

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab08-paper-orders/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Learn the order rules for each session (K5)

Prompt - copy into your AI coding assistant
Using labs_fullcode/pipeline/clock.py, tell me which US session it is now in New York, what that is in Singapore time, and which order types, time_in_force values and extended_hours flag Alpaca will accept right now. Then explain why a market order is not allowed overnight. Do not change any file.

Open in Claude | Open in ChatGPT

Check: You can name the session for a 09:00-18:30 SGT class (OVERNIGHT until 16:00 SGT in US summer time, then PRE) and state 'limit only, extended_hours=True'.

Step 2: Build the planning logic: target vs actual (A3)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/execution.py with a dataclass OrderPlan(symbol, side, qty, current_qty, pending_qty, target_qty, price, reason) and a class OrderManager(broker, cfg, session=None). status() returns pipeline.clock.market_status (the simulator's replay clock if set; the real Alpaca clock online; session=... overrides it offline) and tells the simulator whether the market is open. positions() -> {symbol: qty}; pending() -> signed shares on open orders (+buy, -sell) via GetOrdersRequest(status=OPEN); quote(symbol, session) -> bid/ask/last: online StockLatestQuoteRequest (feed OVERNIGHT first in the overnight session, then IEX), falling back to the latest trade; offline the simulator's last price +/- 0.5 bp. plan(targets, session, units='weight'|'shares'): target_qty = floor(weight x equity / price) (or the share count), delta = target - held - pending, buy if delta > 0, sell (long only, never more than held) if delta < 0, otherwise print 'at target' and send nothing. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: OrderManager(get_broker(state_file='exec_sim.json', reset=True), load_config()).plan({'SPY': 0.30}, 'regular') returns one buy plan for 44 shares.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build session-aware, idempotent order submission (A3, K5)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/execution.py add client_id(plan, tag) = 'lcc-<tag>-<SYMBOL>-<side>-<qty>' (alphanumerics and dashes, max 48 characters; tag = decision date) so the same decision always gets the same id. request(plan, order_type, status, cid, quote, take_profit=0.05, stop_loss=0.03) builds an alpaca-py request: regular + market -> MarketOrderRequest DAY; regular + bracket (buys) -> MarketOrderRequest GTC with OrderClass.BRACKET, TakeProfitRequest and StopLossRequest; closed -> LimitOrderRequest GTC (queues for the next session); pre/after/overnight -> LimitOrderRequest DAY with extended_hours=True. Limit price = ask x (1 + live_paper.limit_buffer_bps) for buys, bid x (1 - buffer) for sells, rounded to 2 decimals. submit(plan, status, tag, order_type='auto'): first broker.get_order_by_client_id(cid) - if it exists print 'DUPLICATE ... not sent again' and return it; print the order (type, TIF, extended_hours, limit, quote and source, cid); send it with broker.submit_order. The broker comes from common.broker.get_broker (Alpaca TradingClient with paper=True, or PaperBrokerSim) - never create a live client. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: Submitting the same plan twice with the same tag prints DUPLICATE the second time.

Step 4: Add polling, protective-order handling and safe fallbacks (A3, K5)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/execution.py add: poll(order, seconds) that re-reads the order every 2 seconds and prints the lifecycle (for example 'new -> filled') and the final order line, or '(still working - it stays on the book)'; cancel_protective(symbol) that cancels open SELL orders and bracket exit legs (GetOrdersRequest nested=True) before an exit, because they reserve the shares; in submit(), if an extended-hours order is rejected, print why (asset or account not enabled for overnight trading, price band, session change) and fall back to a GTC limit order with client id + '-q' that queues for the next session; rebalance(targets, tag, order_type, poll_seconds, units) = status -> plan -> submit -> poll; crypto_demo(broker, cfg) - ONLINE ONLY, clearly labelled DEMO: buy about USD 25 of BTC/USD with a GTC limit at ask + buffer, poll, then sell it back - only to show the fill loop working while US stocks are closed. run(...) and main(argv) for `python -m pipeline execute [--targets SPY=0.30 AAPL=0.10] [--order-type auto|market|limit|bracket] [--session ...] [--live-paper] [--crypto-demo] [--flatten] [--tag] [--reset]`: print [1] the clock and allowed orders, [2] the account, [3] plan -> orders, [4] an idempotency re-run of the same decision plus a deliberate duplicate submission, [5] positions and open orders (with bracket legs), the Alpaca dashboard address when online, and a RESULT line. With --live-paper or online, cap each weight at live_paper.max_position_pct. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline execute --help lists the eight options.

Step 5: Run the order manager in the current session (A3, K5)

Do this (no prompt needed)
Run: python -m pipeline execute --reset (same as: python3 Lab08-paper-orders/execution_stage.py --reset). Offline the session comes from the real clock, so the result depends on when you run it.

Check: Weekend (CLOSED): BUY 44 SPY at LIMIT GTC 678.36 and BUY 36 AAPL at 271.81, both 'accepted' (queued); the re-run prints 'at target (44 shares; held 0, pending +44)' and 'DUPLICATE lcc-<date>-SPY-buy-44'. Weekday Singapore class (OVERNIGHT or PRE): LIMIT DAY extended_hours orders, filled at about 676.63 and 271.12.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 6: Compare the sessions with the teaching override (K5)

Do this (no prompt needed)
Run: python -m pipeline execute --reset --session regular, then --session overnight, then --session closed.

Check: regular: MARKET DAY, filled at 676.70 (SPY) and 271.15 (AAPL), equity USD 99,996.05; overnight: LIMIT DAY extended_hours limit 678.36 / 271.81, filled at 676.63 / 271.12, equity USD 100,000.00; closed: LIMIT GTC, status accepted, 2 open orders. Each run ends with 'no new orders' and one DUPLICATE line.

Step 7: Bracket orders and a clean exit (A3, K5)

Do this (no prompt needed)
Run: python -m pipeline execute --reset --session regular --order-type bracket. Then flatten: python -m pipeline execute --flatten --session regular.

Check: Bracket legs: SPY take-profit 710.47 / stop 656.34, AAPL 284.68 / 262.99 (status new). Flatten prints 'cancelled 2 protective exit order(s)' for each symbol, sells 44 SPY at 676.57 and 36 AAPL at 271.09, equity USD 99,992.09, 0 open orders.

Step 8: Trade LIVE on the Alpaca paper account during class (A3, K5)

Do this (no prompt needed)
With paper keys in .env run: python -m pipeline execute --live-paper --targets SPY=0.05. Watch the clock line, the quote, the extended-hours limit order and its lifecycle for up to 30 seconds. Then open the Alpaca paper dashboard (app.alpaca.markets > Paper > Orders and Positions) and find the same client order id.

Check: Clock shows OVERNIGHT or PRE; one LIMIT DAY extended_hours buy priced at ask + 25 bps; lifecycle 'new -> filled' (or still working); the order and position appear in the dashboard. If Alpaca rejects it, the stage explains why and queues a GTC limit order instead.

Step 9: Optional - watch a fill right now with the crypto demo (K5)

Do this (no prompt needed)
If US stocks are closed (weekend) or your extended-hours order is still working, run: python -m pipeline execute --live-paper --targets SPY=0.05 --crypto-demo. This is a DEMO of the fill loop only; the course and the pipeline remain stock-focused.

Check: A BTC/USD buy of about USD 25 and the matching sell, each GTC limit, filled within seconds and visible in the dashboard.

Step 10: Flatten the live paper account (A3)

Do this (no prompt needed)
Run: python -m pipeline execute --live-paper --flatten, then refresh the dashboard. Offline, run the same command without --live-paper.

Check: Protective orders are cancelled, sell orders are sent (extended-hours limits outside regular hours) and the positions list empties once they fill.

Run it

Step	Command / file
Run Lab 08	python3 Lab08-paper-orders/execution_stage.py (same as: python -m pipeline execute)
The complete code	pipeline/execution.py (reference copy: Lab08-paper-orders/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.